

Universidad Santo Tomás de Aquino (UNSTA)

Facultad de Ingeniería

INFORME TÉCNICO FINAL

Diseño y Evolución Arquitectónica de E-Commerce Multivendedor Cloud-Native

Materia: Desarrollo de Aplicaciones Web

Proyecto: Código-Cuatro

Alumnos:

- Benjamín Lopez Zigarán
- Facundo Nosa
- Juan Mignone
- Juan Pablo Valdez

Año Académico: 2026

Índice

1. Introducción 1.1. Contexto de Negocio 1.2. Problema Resuelto 1.3. Alcance del Documento
2. Fase 1: Arquitectura Base y Esqueleto (El MVP) 2.1. Modelo de Negocio y Requerimientos Iniciales 2.2. Diseño Arquitectónico del Monolito 2.3. Diagrama de Arquitectura Inicial 2.4. Justificación Técnica y de Negocio 2.5. Límites y Cuellos de Botella 2.6. Estrategia de Ramas y Flujo de Trabajo (GitFlow)
3. Fase 2: Evolución Forzada a Microservicios 3.1. El Cambio de Contexto (Escenario de Negocio) 3.2. Paso A: Microservicios Tradicionales (Síncronos) 3.3. Paso B: Microservicios Modernos (Event-Driven / Asíncronos) 3.4. Diagramas de Arquitectura Comparativos
4. Fase 3: Mejora Incremental (Robustez y Escalabilidad) 4.1. Estrategia de Caché de Alta Performance (Redis) 4.2. Replicación de Base de Datos (Read Replicas en PostgreSQL) 4.3. Capa de Balanceo de Carga e Infraestructura de Red 4.4. Políticas Matemáticas de Auto-scaling Horizontal (Kubernetes HPA)
5. Fase 4: Infraestructura Cloud-Native y DevOps (*Próximamente*)
6. Fase 5: Conclusiones (*Próximamente*)
7. Anexos (*Próximamente*)

1. Introducción

1.1. Contexto de Negocio

El proyecto "Codigo-Cuatro" nace con el propósito de resolver una necesidad crítica en el ecosistema comercial local: la falta de una plataforma de comercio electrónico accesible, moderna y centralizada que permita a múltiples vendedores (comercios locales) exponer su catálogo, gestionar su inventario y administrar sus pedidos dentro de un único ecosistema digital. Se plantea como una solución "multi-vendor" (multivendedor), en la cual los clientes finales pueden explorar productos de diversos orígenes y concretar compras a través de un flujo unificado, mientras que los comercios mantienen autonomía sobre sus operaciones.

1.2. Problema Resuelto

Actualmente, los pequeños y medianos comercios enfrentan altas barreras de entrada para la digitalización de sus ventas, debiendo recurrir a plataformas genéricas que diluyen su identidad o afrontar el alto costo de desarrollar soluciones a medida. "Codigo-Cuatro" resuelve esta fricción al proveer un entorno de E-commerce donde la mediación digital entre inventario y consumidores está abstraída. La plataforma se encarga de la gestión de la infraestructura, seguridad, autorización y flujos de pago, permitiendo a los comerciantes enfocarse exclusivamente en la gestión de su catálogo y logística de pedidos.

1.3. Alcance del Documento

El presente Informe Técnico Final detalla el ciclo de vida completo del diseño arquitectónico del proyecto "Codigo-Cuatro" para la materia Desarrollo de Aplicaciones Web. Lejos de ser un sistema estático, se aborda el proyecto desde una perspectiva evolutiva, demostrando cómo la arquitectura de software debe acompañar el crecimiento del negocio.

El alcance de este documento abarca desde la Fase 1, que define el Producto Mínimo Viable (MVP) construido sobre una arquitectura monolítica modular, pasando por la transición hacia microservicios síncronos y asíncronos (event-driven), la implementación de estrategias de alta disponibilidad, hasta culminar en una infraestructura Cloud-Native gestionada mediante Infraestructura como Código (IaC) en AWS. Cada decisión arquitectónica a lo largo de estas fases se someterá a un riguroso análisis y contará con su respectiva justificación técnica y de negocio acorde a un nivel avanzado de ingeniería de software.

2. Fase 1: Arquitectura Base y Esqueleto (El MVP)

2.1. Modelo de Negocio y Requerimientos Iniciales

Para la concepción del Producto Mínimo Viable (MVP), se estipularon escenarios de uso conservadores alineados con un lanzamiento inicial (Go-to-Market):

- **Comercios registrados:** 5 a 20.
- **Cientes registrados:** 100 a 500.
- **Usuarios concurrentes:** 10 a 50.
- **Pedidos diarios:** 20 a 100.

El requerimiento principal en esta etapa era validar el modelo de negocio con la menor complejidad operativa posible, garantizando al mismo tiempo una separación clara de responsabilidades (Frontend, Backend, Persistencia e Infraestructura) que no comprometa el crecimiento futuro.

2.2. Diseño Arquitectónico del Monolito

La Fase 1 se diseñó bajo un patrón de **Monorepo con frontend y backend independientes**, apoyado en un backend monolítico estructurado modularmente en capas. Esta decisión técnica consolida los siguientes componentes principales:

- **Frontend (web/):** Desarrollado como una Single Page Application (SPA) utilizando **React 19 y Vite**. Se implementó un enrutamiento basado en **TanStack Router** para manejar y estructurar eficientemente las vistas públicas, administrativas y de vendedores. La gestión del estado del servidor se delega a **TanStack Query**, fundamental para el cacheo, optimización y sincronización rápida del catálogo y carrito. Por otro lado, **Zustand** maneja el estado local ligero (como la sesión de usuario activa). Todo el diseño visual se apoya en los estándares modernos proveídos por **shadcn/ui** y **Tailwind CSS**.
- **Backend (api/):** Centralizado en un único servicio monolítico construido sobre **NestJS 11**. Su arquitectura interna respeta una estricta separación de responsabilidades en tres capas fundamentales:
 - **Capa de Transporte:** Implementada a través de Controladores (Controllers) encargados de la recepción de peticiones HTTP, validación en tiempo de ejecución de DTOs (Data Transfer Objects) mediante **class-validator**, y el correspondiente versionado semántico de la API (ej. **/v1/**).
 - **Capa de Aplicación:** Implementada a través de Servicios (Services) donde reside toda la lógica de negocio imperativa, cálculos de costos, validaciones de dominio y orquestación entre módulos aislados funcionalmente (como Auth, Users, Companies, Catalog, Inventory y Orders).
 - **Capa de Datos:** Integración exclusiva a través de **Prisma ORM**, abstrayendo las consultas de la base de datos, asegurando validación de tipos a nivel de transacciones y evitando la diseminación de sentencias SQL no seguras.
- **Persistencia:** Centralizada en una única instancia relacional de **PostgreSQL 16**.
- **Infraestructura (infra/):** Definición temprana de recursos en la nube (AWS) implementando Infraestructura como Código mediante **Terraform**. Se priorizan servicios administrados como Amazon RDS (base de datos relacional) y Amazon Cognito (gestión de identidad).

2.3. Diagrama de Arquitectura Inicial

A continuación, se ilustra la topología del sistema durante la Fase 1, evidenciando el flujo de comunicación desde el cliente web hasta la capa de persistencia en la infraestructura.

```

flowchart TD
    %% Usuarios
    U[Usuario Web  
Cliente / Vendedor / Admin]

    %% Frontend Independiente
    subgraph Frontend [Aplicación Frontend]
        W[web/  
React 19 + Vite  
TanStack Router  
TanStack Query + Zustand]
    end

    %% Backend Monolítico
    subgraph Backend [Backend API Monolítica NestJS]
        direction TB
        T[Capa de Transporte  
Controllers HTTP /v1/]
        A[Capa de Aplicación  
Services + Reglas de Negocio]
        D[Capa de Datos  
Prisma ORM]
    end

    T --> A
    A --> D

    %% Base de datos compartida
    subgraph DB [Persistencia]
        P[(PostgreSQL 16)]
    end

    %% Infraestructura como Código
    subgraph Infra [Infraestructura AWS]
        I[infra/  
Terraform + Servicios Administrados]
    end

    %% Relaciones y Flujos
    U -->|Interacción SPA| W
    W -->|Peticiones Síncronas HTTP / REST| T
    D -->|Queries SQL| P

```

2.4. Justificación Técnica y de Negocio

JUSTIFICACIÓN TÉCNICA Y DE NEGOCIO

Decisión Arquitectónica: Implementar un Monolito Modular con PostgreSQL único en lugar de una arquitectura de Microservicios para el MVP.

Perspectiva de Negocio: Para validar la hipótesis inicial del e-commerce multivendedor (alcanzar el Product-Market Fit), la velocidad de desarrollo e iteración (Time-to-Market) se impone como el factor más crítico. El costo operativo, cognitivo y económico de provisionar e instrumentar múltiples microservicios distribuidos desde el día cero es prohibitivo para un sistema que, de acuerdo a las estimaciones iniciales, operará con apenas 5 a 20 comercios. Una arquitectura monolítica permite agregar funcionalidades rápidamente y reducir drásticamente los gastos de infraestructura Cloud (AWS) durante las etapas tempranas del proyecto, priorizando la viabilidad financiera.

Perspectiva Técnica: Al optar por **NestJS** como framework backend, se adoptó una disciplina de desarrollo estricta forzando el uso de módulos altamente cohesivos y débilmente acoplados (por ejemplo, manteniendo estricta separación entre dominios como Catalog e Inventory) a pesar de convivir en el mismo repositorio y ejecutarse en el mismo proceso del sistema operativo. Esta modularidad interna previene proactivamente la entropía arquitectónica conocida como "Big Ball of Mud" o "Código Espagueti" y establece fronteras de dominio sumamente claras, allanando el camino técnico para una extracción directa y sin fricciones hacia verdaderos microservicios distribuidos en el futuro. Paralelamente, delegar toda la persistencia a un clúster central de **PostgreSQL** garantiza por diseño propiedades **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad); una característica absolutamente indispensable para transaccionar órdenes, pagos y descuentos de stock concurrentes sin enfrentarse prematuramente a la enorme complejidad de manejar patrones de consistencia eventual a través de múltiples bases de datos.

2.5. Límites y Cuellos de Botella

Aunque la arquitectura monolítica descrita se presenta como la solución óptima para la fase inicial de validación del proyecto, por su propia naturaleza centralizada asume deudas técnicas y presenta límites estructurales marcados que eventualmente, frente a un incremento sostenido de demanda, motivarán su inevitable evolución arquitectónica:

1. **Escalabilidad Acoplada e Ineficiente:** En escenarios donde una campaña publicitaria masiva dispare las visualizaciones de productos (alta demanda intensiva de lecturas sobre el dominio del Catálogo), la plataforma exigirá la replicación completa de toda la API. Este acoplamiento provocará que módulos inactivos o de baja carga, como los de administración o generación de reportes, consuman recursos computacionales (CPU/RAM) innecesarios, inflando exponencialmente los costos operacionales.
2. **Riesgo Sistémico por Falla Local (Single Point of Failure):** Al cohabitar todas las subrutinas de la aplicación dentro de un mismo bloque de memoria, un fallo crítico o un loop infinito introducido en un módulo periférico o de soporte —por ejemplo, durante la generación asíncrona de facturas PDF— es capaz de inducir un colapso de memoria (**Out Of Memory**) que aborte el hilo principal de Node.js, comprometiendo instantáneamente operaciones esenciales e irrecuperables en otros dominios, tales como el procesamiento en tiempo real de pasarelas de pagos.
3. **Saturación y Contención en la Capa de Datos:** La dependencia central hacia una única instancia de PostgreSQL implica que todos los dominios internos de la aplicación deben competir por el acceso a un mismo límite de conexiones (Connection Pool). Tareas de gran intensidad transaccional (ej. cierres contables masivos de vendedores o actualizaciones masivas de precios) generarán un bloqueo temporal que degradará severamente los tiempos de respuesta del catálogo para los usuarios finales. Adicionalmente, el esquema de datos monolítico previene la selección del motor de base de datos más óptimo para cada necesidad específica (por ejemplo, usar Redis para caché de catálogo o NoSQL para historizar eventos).

2.6. Estrategia de Ramas y Flujo de Trabajo (GitFlow)

Para asegurar un control riguroso del ciclo de vida del software y coordinar el trabajo simultáneo sobre el monorepo inicial, se adoptó **GitFlow** como estrategia formal de ramificación. Esta elección operativa establece una base sólida para el trabajo en equipo y la futura automatización.

Diagrama de Flujo de Ramas

```
gitGraph
  commit id: "Initial"
  branch develop
  checkout develop
  commit id: "Setup"
  branch feature/catalog
  checkout feature/catalog
  commit id: "Add Catalog"
  checkout develop
  merge feature/catalog
  branch release/v1.0.0
  checkout release/v1.0.0
  commit id: "QA Fixes"
  checkout main
  merge release/v1.0.0 tag: "v1.0.0"
  checkout develop
  merge release/v1.0.0
  checkout main
  branch hotfix/auth-bug
  checkout hotfix/auth-bug
  commit id: "Fix Auth"
  checkout main
  merge hotfix/auth-bug tag: "v1.0.1"
  checkout develop
  merge hotfix/auth-bug
```

JUSTIFICACIÓN TÉCNICA Y DE NEGOCIO

Decisión Operativa: Adoptar GitFlow sobre Trunk-Based Development o GitHub Flow.

Perspectiva de Negocio: Garantiza que el código que alcanza la rama de producción (**main**) haya superado un proceso de estabilización y pruebas, minimizando el riesgo de disrupción en el servicio de e-commerce que afectaría tanto a clientes como a vendedores. La capacidad de emitir **hotfixes** directos permite responder a incidentes críticos (ej. caída de pasarela de pago) en minutos sin detener el trabajo de las nuevas funcionalidades.

Perspectiva Técnica: Al contar con múltiples ambientes físicos definidos (Dev, QA, UAT, Prod), la estructura semántica de GitFlow se mapea de manera 1:1 con estos entornos. **develop** sirve como punto de integración (QA), **release/*** actúa como un entorno de estabilización (UAT), y **main** es el estado inmutable de Producción. Además, el uso de ramas **feature/*** asegura que el desarrollo de distintos dominios (Auth, Orders, Inventory) se mantenga aislado, reduciendo drásticamente los conflictos de código (merge conflicts) en el monorepo.

3. Fase 2: Evolución Forzada a Microservicios

3.1. El Cambio de Contexto (Escenario de Negocio)

A pesar del éxito del MVP construido sobre una arquitectura monolítica, el crecimiento comercial superó las proyecciones iniciales de manera abrupta. La adopción temprana catapultó la plataforma a **487 comercios activos, 14.300 clientes registrados y picos de demanda superiores a los 820 pedidos por hora** (típicamente durante eventos de alta concurrencia como un "Cyber Monday" local).

Este escenario de hipercrecimiento destapó las limitaciones arquitectónicas pronosticadas:

- **Lentitud Crítica en Checkout:** El módulo de órdenes comenzó a competir por el mismo pool de conexiones a la base de datos PostgreSQL que el módulo del catálogo (el cual recibía miles de lecturas por segundo).
- **Inconsistencias de Inventario:** Picos de demanda concurrente sobre productos populares resultaron en errores de sobreventa por falta de aislamiento transaccional avanzado.
- **Cuellos de Botella en Despliegue:** Con un equipo creciente trabajando sobre el mismo repositorio, los despliegues monolíticos generaban indisponibilidad de toda la API, interrumpiendo transacciones en curso.

Esta situación demandó abandonar el monolito de forma imperativa para poder escalar equipos y recursos computacionales (CPU/RAM) de manera independiente por dominio.

3.2. Paso A: Microservicios Tradicionales (Síncronos)

Diseño de la Arquitectura

La primera etapa evolutiva consistió en descomponer el monolito en 9 microservicios autónomos (Auth, User, Catalog, Inventory, Order, Payment, Notification, Storage y Admin). Cada servicio se erigió como una aplicación NestJS independiente, comunicada con las demás a través de llamadas **HTTP/REST síncronas**.

Crucialmente, se adoptó el patrón **Database per Service**. Cada microservicio obtuvo su propia base de datos PostgreSQL, garantizando un acoplamiento nulo a nivel de persistencia de datos (evitando que un servicio bloquee tablas de otro).

Patrones Incorporados

- **API Gateway:** Se introdujo como un punto único de entrada para todas las peticiones del frontend. Sus responsabilidades incluyen el enrutamiento por prefijo (ej. `/orders` a `order-service`), la validación primaria de tokens JWT y limitación de tasa (Rate Limiting).
- **Circuit Breaker:** Dado que las llamadas eran síncronas (ej. el `order-service` esperando respuesta del `payment-service`, y este último esperando respuesta del proveedor externo de tarjetas), se implementó el patrón Circuit Breaker mediante la biblioteca `opossum`. Si un servicio aguas abajo falla o excede el timeout, el circuito se "abre", cortando la comunicación instantáneamente para evitar el agotamiento de hilos (thread exhaustion) y fallos en cascada a través de toda la infraestructura.

Justificación y Límites

JUSTIFICACIÓN TÉCNICA Y DE NEGOCIO

Decisión Arquitectónica: Migrar a Microservicios Síncronos con patrón Database-per-Service.

Perspectiva de Negocio: Esta descomposición permite que el tráfico masivo de lectura del Catálogo (usuarios mirando productos sin comprar) no consuma la capacidad computacional de la pasarela de pagos. Cada equipo de desarrollo puede lanzar nuevas versiones de su servicio sin coordinar un despliegue global ("zero-downtime deployment"), acelerando el Time-to-Market de nuevas funcionalidades.

Perspectiva Técnica: Adoptar REST síncrono fue el camino de menor resistencia para refactorizar el monolito, ya que la semántica request/response se mantuvo casi idéntica. El API Gateway ocultó la topología distribuida al frontend, facilitando la transición.

Límites: El principal límite de esta etapa es el **Acoplamiento Temporal**. El proceso de Checkout se transformó en una larga cadena síncrona (**Order** -> **Inventory** -> **Payment** -> **Notification**). Si la pasarela de pago experimenta lentitud (ej. suma 3 segundos), el usuario final percibe esos 3 segundos más la latencia de red en cada salto HTTP. Peor aún, si **Notification** falla después de confirmarse el pago, la transacción global queda en un estado parcialmente inconsistente al no existir transacciones ACID distribuidas reales.

3.3. Paso B: Microservicios Modernos (Event-Driven / Asíncronos)

Evolución Arquitectónica

Para resolver la fragilidad y el acoplamiento temporal del Paso A, el sistema evolucionó hacia una **Arquitectura Orientada a Eventos (EDA)**. Se erradicó la comunicación REST síncrona en el flujo crítico (checkout) y se delegó la coordinación a un **Message Broker** robusto, gestionado en la nube: AWS Simple Notification Service (SNS) emparejado con Amazon Simple Queue Service (SQS).

Casos de Uso de Eventos (El Nuevo Checkout)

El proceso de compra pasó a ser completamente asíncrono:

1. El usuario envía su orden, el **order-service** la persiste en estado **PENDING** y devuelve un código HTTP **202 Accepted** de inmediato (< 200ms de latencia percibida).
2. El **order-service** publica el evento **OrderCreated** en el tópico central de SNS.
3. El **inventory-service** (suscrito vía SQS) consume el evento en segundo plano, reserva el stock y publica **StockReserved**.
4. El **payment-service** consume este último, procesa el cobro y publica **PaymentAuthorized**.
5. El **notification-service** y el **order-service** reaccionan a este evento final para enviar emails y marcar la orden como **CONFIRMED**.

Patrones Avanzados

- **Saga (Coreografía):** Se implementó este patrón para manejar transacciones distribuidas sin un coordinador central. Si, por ejemplo, el proveedor rechaza el pago, el **payment-service** publica el evento **PaymentFailed**. De forma autónoma, el **inventory-service** consume este error y ejecuta su acción compensatoria (liberar el stock reservado), mientras que **order-service** marca el pedido como **CANCELLED**.

- **CQRS (Command Query Responsibility Segregation):** Al separar el modelo de escritura del de lectura, el **catalog-service** consume eventos como **ProductUpdated** y mantiene una vista materializada (Read Model) en su base de datos propia, optimizada al 100% para búsquedas y evitando realizar joins costosos en tiempo de ejecución.

JUSTIFICACIÓN TÉCNICA Y DE NEGOCIO

Decisión Arquitectónica: Adoptar Event-Driven Architecture con AWS SQS/SNS, Saga y CQRS.

Perspectiva de Negocio: La latencia de compra (el tiempo desde que el cliente pulsa "Pagar" hasta que la interfaz responde) se reduce a fracciones de segundo, garantizando una excelente experiencia de usuario (UX) que impacta positivamente en la tasa de conversión durante eventos de tráfico masivo. Además, al delegar la infraestructura de mensajería a AWS (Serverless), se evita la costosa contratación de ingenieros especializados en administrar clústeres auto-gestionados (como Kafka o RabbitMQ).

Perspectiva Técnica: Las colas SQS actúan como un buffer o "amortiguador" (*Shock Absorber*). Si el servicio de notificaciones colapsa o el proveedor de tarjetas de crédito se ralentiza, el sistema no se cae; los eventos simplemente se encolan y se procesan cuando la capacidad se restablece (*Backpressure*). Las colas de mensajes no entregados (*Dead Letter Queues*) garantizan que ninguna transacción se pierda de forma silenciosa.

3.4. Diagramas de Arquitectura Comparativos

Diagrama 2.A: Microservicios Tradicionales (Síncronos)

```
graph TD
    %% Convenciones
    classDef client fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;
    classDef gateway fill:#ffe0b2,stroke:#f57c00,stroke-width:2px;
    classDef service fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
    classDef database fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px;
    classDef external fill:#eceff1,stroke:#455a64,stroke-width:2px;

    Client[Cliente / Frontend SPA]:::client -->|HTTP/REST| Gateway[API Gateway NestJS]:::gateway

    subgraph "Comunicaciones REST Síncronas (Acoplamiento Temporal)"
        Gateway -->|Enrutamiento| CatalogSvc[catalog-service]:::service
        Gateway -->|Enrutamiento| OrderSvc[order-service]:::service
        Gateway -->|Enrutamiento| AuthSvc[auth-service]:::service

        OrderSvc -->|1. GET /products/:id| CatalogSvc
        OrderSvc -->|2. POST /reserve| InventorySvc[inventory-service]:::service
        OrderSvc -->|3. POST /process| PaymentSvc[payment-service]:::service
        OrderSvc -->|4. POST /send| NotificationSvc[notification-service]:::service
    end

    subgraph "Patrón Database per Service"
        CatalogSvc --- DB_Catalog[(PostgreSQL db_catalog)]:::database
    end
```

```

    OrderSvc --- DB_Order[(PostgreSQL
db_orders)]:::database
    InventorySvc --- DB_Inv[(PostgreSQL
db_inventory)]:::database
    PaymentSvc --- DB_Pay[(PostgreSQL
db_payments)]:::database
end

PaymentSvc -->|Circuit Breaker (Opossum)| ExtPayment[Proveedor Externo de
Pagos]:::external

```

Diagrama 2.B: Microservicios Modernos (Event-Driven)

```

graph TD
    %% Convenciones
    classDef client fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;
    classDef gateway fill:#ffe0b2,stroke:#f57c00,stroke-width:2px;
    classDef service fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
    classDef database fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px;
    classDef broker fill:#fff9c4,stroke:#fbc02d,stroke-width:2px;
    classDef external fill:#eceff1,stroke:#455a64,stroke-width:2px;
    classDef readmodel fill:#e0f7fa,stroke:#0097a7,stroke-width:2px;

    Client[Cliente / Frontend SPA]:::client -->|HTTP/REST Asíncrono| Gateway[API
Gateway]:::gateway

    Broker((AWS SNS / SQS
Message Broker)):::broker

    subgraph "Saga Coreografiada (Desacoplada)"
        Gateway -->|Enrutamiento| OrderSvc[order-service]:::service
        OrderSvc -. ->|1. Pub: order.created| Broker

        Broker -. ->|2. Sub: order.created| InventorySvc[inventory-
service]:::service
        InventorySvc -. ->|3. Pub: stock.reserved / stock.reservation_failed|
Broker

        Broker -. ->|4. Sub: stock.reserved| PaymentSvc[payment-service]:::service
        PaymentSvc -. ->|5. Pub: payment.processed / payment.failed| Broker

        Broker -. ->|6. Sub: payment.processed / failed| OrderSvc
        OrderSvc -. ->|7. Pub: order.confirmed / cancelled| Broker

        Broker -. ->|8. Sub: order.confirmed / cancelled|
NotificationSvc[notification-service]:::service
    end

    subgraph "CQRS y Persistencia"
        OrderSvc --- DB_Order[(PostgreSQL
db_orders)]:::database
    end

```

```
    InventorySvc --- DB_Inv[(PostgreSQL
db_inventory)]:::database
    PaymentSvc --- DB_Pay[(PostgreSQL
db_payments)]:::database

    Gateway -->|Lecturas Optimizadas| CatalogSvc[catalog-service
CQRS Read Model]:::readmodel
    Broker -->|9. Sub: product.updated| CatalogSvc
    CatalogSvc --- DB_CatalogRM[(PostgreSQL
db_catalog Read Model)]:::database
    end

    PaymentSvc -->|Circuit Breaker Residual| ExtPayment[Proveedor Externo de
Pagos]:::external
```

4. Fase 3: Mejora Incremental (Robustez y Escalabilidad)

4.1. Estrategia de Caché de Alta Performance (Redis)

Para mitigar la carga sobre las bases de datos y reducir la latencia general del sistema frente a un tráfico elevado, se incorporó **Redis** como motor de caché en memoria de alta disponibilidad (Redis Cluster).

Ubicación y Rol Estratégico:

- **catalog-service**: Implementa un patrón **Cache-Aside**. Dado que miles de usuarios leen el catálogo concurrentemente, las peticiones GET se resuelven contra Redis en tiempos inferiores a 3ms, eludiendo la necesidad de consultar a PostgreSQL, exceptuando los "cache misses".
- **auth-service**: Utiliza Redis para validar el estado de los tokens JWT de sesión (permitiendo forzar deslogues centralizados antes de la expiración criptográfica).
- **api-gateway**: Utiliza los contadores atómicos rápidos de Redis (**INCR** y **EXPIRE**) para implementar políticas estrictas de *Rate Limiting* por IP y prevenir ataques de denegación de servicio o abuso de la API.

Políticas de Invalidación (Consistencia): Para evitar el problema de la "inconsistencia eventual prolongada", la caché no depende únicamente de su Tiempo de Vida (TTL). La arquitectura se acopla a la red asíncrona de la Fase 2: cuando un vendedor actualiza un producto, se publica el evento **product.updated** en el Message Broker. El **catalog-service** consume este evento e invalida de forma explícita (**DEL**) las claves afectadas en Redis. Esto asegura que el frontend reciba la versión más reciente del dato casi instantáneamente, preservando la consistencia del catálogo.

JUSTIFICACIÓN TÉCNICA Y DE NEGOCIO

Decisión Arquitectónica: Implementar caché distribuida en Redis con invalidación impulsada por eventos.

Perspectiva de Negocio: Una página de producto que carga en 200ms en lugar de 1.5s incrementa dramáticamente la conversión de ventas. Redis permite sostener los picos del tráfico durante campañas publicitarias sin necesidad de escalar horizontalmente el costoso motor relacional de PostgreSQL.

Perspectiva Técnica: Redis se eligió por encima de Memcached por su soporte nativo de estructuras complejas y comandos atómicos necesarios para el Rate Limiting. Acoplar la invalidación al flujo Event-Driven garantiza que el sistema siga mostrando datos frescos sin requerir complejas lógicas de barrido manual.

4.2. Replicación de Base de Datos (Read Replicas en PostgreSQL)

En paralelo a la capa de caché, la persistencia subyacente evolucionó hacia una topología **Master-Replica (Multi-AZ en AWS RDS)**.

División de Operaciones:

- **Nodos Master (Escritura):** Servicios críticos y de alta mutabilidad como el **order-service** y el **inventory-service** dirigen el 100% de sus transacciones SQL (**INSERT**, **UPDATE**, **DELETE**) hacia el nodo principal. Esto asegura consistencia ACID estricta al procesar pagos y reservas de stock.

- **Nodos Read Replica (Lectura):** El *Read Model* del `catalog-service` (patrón CQRS) y los módulos de reportes analíticos dirigen sus extensas consultas (`SELECT`) hacia instancias secundarias de solo lectura. Estas réplicas se mantienen sincronizadas asincrónicamente con el Master por el propio motor de PostgreSQL.

JUSTIFICACIÓN TÉCNICA Y DE NEGOCIO

Decisión Arquitectónica: Segregación de tráfico de base de datos mediante Read Replicas.

Perspectiva de Negocio: Permite extraer costosos reportes de ventas mensuales o de inteligencia de negocio en tiempo real sin degradar ni un milisegundo el proceso de "checkout" de un cliente activo.

Perspectiva Técnica: La replicación aísla las cargas de lectura masiva de los bloqueos de escritura transaccional. Además, en caso de caída catastrófica del nodo Master, AWS RDS promociona automáticamente una réplica a Master, garantizando una Alta Disponibilidad (High Availability) con un RTO (Recovery Time Objective) de apenas minutos.

4.3. Capa de Balanceo de Carga e Infraestructura de Red

Para manejar eficientemente una carga simulada y certificada de **210 RPS (Requests Per Second)** y **1.400 usuarios concurrentes**, la topología de red se dividió en dos fronteras distintas mediante Kubernetes:

- **Balanceo Híbrido (ALB L7 y ClusterIP L4):**
 - Se aprovisionó un **Application Load Balancer (ALB)** de AWS operando en **Capa 7**. Su rol es gestionar la terminación de certificados TLS/SSL (descargando a los contenedores de este trabajo criptográfico pesado) y enrutar inteligentemente el tráfico basado en "paths" (ej. `/api/*`) directo al `api-gateway`.
 - Internamente, la comunicación inter-microservicios utiliza objetos `Service` nativos de Kubernetes de tipo `ClusterIP` operando a nivel de **Capa 4 (TCP)**. Esto proporciona un balanceo de alta velocidad y latencia nula sin la penalización de re-evaluar headers HTTP en cada salto interno.
- **Algoritmos de Distribución:**
 - **Round Robin:** Utilizado para servicios de cómputo uniforme como el `catalog-service`, donde el costo computacional de devolver un JSON de producto es idéntico entre peticiones.
 - **Least Connections:** Utilizado para servicios transaccionales con tiempos de procesamiento sumamente asimétricos y variables (ej. `payment-service` aguardando la respuesta impredecible de una pasarela externa), evitando que un contenedor lento acumule demasiadas peticiones en espera.
- **Sondeo de Salud (Health Checks y Mitigación de Warm-up):** Se instrumentaron sondas nativas con NestJS Terminus. La sonda de **Readiness** asume un rol crítico: Kubernetes no enviará tráfico a un Pod recién creado hasta que este confirme que su *pool* de conexiones a PostgreSQL y Redis está 100% establecido. Esto elimina los clásicos errores `502 Bad Gateway` causados por balancear tráfico hacia aplicaciones cuyo "warm-up" (inicialización) aún no concluyó.

JUSTIFICACIÓN TÉCNICA Y DE NEGOCIO

Decisión Arquitectónica: Balanceo de doble capa (ALB L7 externo + ClusterIP L4 interno) y sondas estrictas de Readiness.

Perspectiva de Negocio: Garantiza "Zero-Downtime Deployments". Los clientes nunca percibirán un corte de servicio mientras el equipo técnico publica nuevas versiones, protegiendo la reputación de la marca durante horas pico de ventas.

Perspectiva Técnica: Al procesar SSL en el borde (ALB), liberamos capacidad de CPU en el clúster. La mitigación del "warm-up" de Node.js es fundamental, ya que el motor v8 requiere unos segundos iniciales para compilar y optimizar el código (JIT) e hidratar conexiones; enviar peticiones antes de esto resultaría en latencias altísimas para esos primeros usuarios.

4.4. Políticas Matemáticas de Auto-scaling Horizontal (Kubernetes HPA)

Se prohibió el sobre-aprovisionamiento estático de contenedores, adoptando políticas matemáticas estrictas de auto-escalado impulsadas por el **Horizontal Pod Autoscaler (HPA)** para gestionar las 210 RPS.

El estándar de diseño arquitectónico estipula un umbral base de **70% de utilización de CPU para "Scale-out" (crecer) y 30% para "Scale-in" (reducir)**.

- **Justificación del Headroom de Seguridad:** Escalar al 70% deja un 30% de "colchón" (headroom) de CPU libre. Durante el tiempo de espera en el cual Kubernetes descarga la imagen Docker y arranca la aplicación (de 30 a 60 segundos), los Pods existentes utilizarán este colchón para absorber de forma transitoria el pico violento de tráfico. Si el umbral fuera del 90%, el servicio colapsaría en estrangulamiento térmico o paradas del Garbage Collector antes de que los refuerzos lleguen.
- **Justificación del Scale-in (30%) y Cooldown:** El umbral bajo (30%) y la ventana de estabilización (cooldown) de 5 minutos evitan el *thrashing* o "flapping". Impide que Kubernetes destruya máquinas ante una pausa de un minuto en el tráfico, solo para volver a crearlas desesperadamente un minuto después, lo cual infla severamente la factura de la nube.

Tabla de Políticas HPA por Servicio

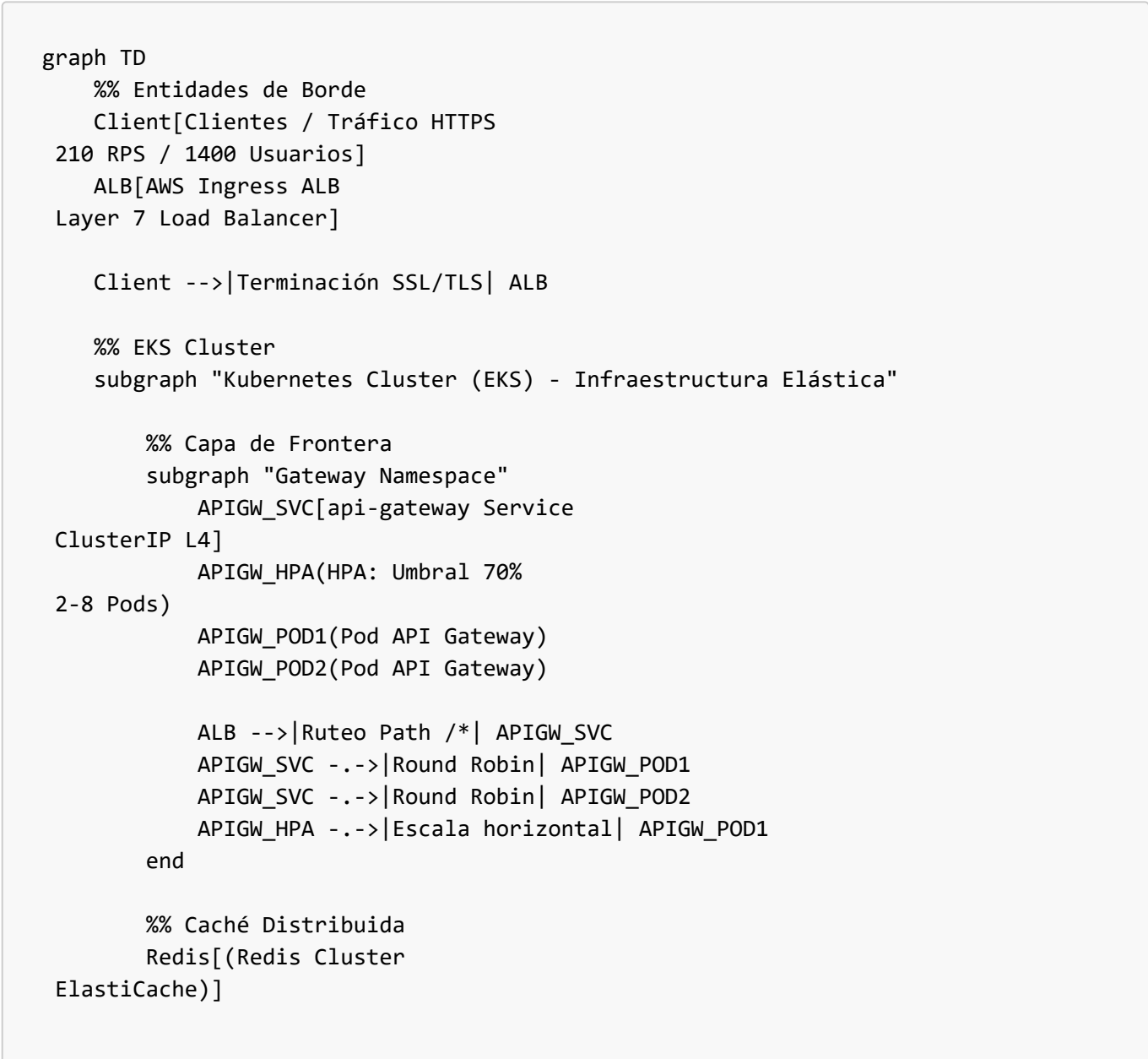
Microservicio	Métrica Disparadora (K8s HPA)	Umbral Scale-out	Umbral Scale-in	Cooldown (Scale-out/in)	Mínimo Réplicas	Máximo Réplicas
api-gateway	TargetCPUUtilizationPercentage	70%	30%	300 segundos	2	8
catalog-service	TargetCPUUtilizationPercentage	70%	30%	300 segundos	3	10
inventory-service	TargetCPUUtilizationPercentage	75%	35%	300 segundos	2	6
order-service	TargetCPUUtilizationPercentage	65%	30%	300 segundos	2	8
user-service	TargetCPUUtilizationPercentage	75%	35%	300 segundos	2	5

Microservicio	Métrica Disparadora (K8s HPA)	Umbral Scale-out	Umbral Scale-in	Cooldown (Scale-out/in)	Mínimo Réplicas	Máximo Réplicas
payment-service	TargetCPUUtilizationPercentage	60%	25%	300 segundos	2	6
auth-service	TargetCPUUtilizationPercentage	70%	30%	300 segundos	2	8

(Nota: Los servicios críticos dependientes de factores externos o coordinación asíncrona, como *payment-service* u *order-service*, poseen umbrales de alerta temprana del 60-65% para garantizar el aprovisionamiento preventivo).

4.5. Diagrama de Arquitectura Integrado

El siguiente esquema integra toda la evolución de la Fase 3, consolidando la capa de red pública, el balanceo interno en Kubernetes (EKS), las políticas de escalado elástico, el clúster Redis de alta disponibilidad y la topología asimétrica Master-Replica de la base de datos relacional.




```

    %% Servicios Críticos Internos
    subgraph "Catalog Namespace"
        CAT_SVC[catalog-service
ClusterIP L4]
        CAT_HPA(HPA: Umbral 70%
3-10 Pods)
        CAT_POD(Replica Set
Catalog Pods)

        APIGW_POD1 -->|gRPC / HTTP L4| CAT_SVC
        APIGW_POD2 -->|gRPC / HTTP L4| CAT_SVC
        CAT_SVC -->|Round Robin| CAT_POD
        CAT_HPA -->|Escala horizontal| CAT_POD
        CAT_POD -->|Cache-Aside (Hit/Miss)| Redis
    end

    subgraph "Order Namespace"
        ORD_SVC[order-service
ClusterIP L4]
        ORD_HPA(HPA: Umbral 65%
2-8 Pods)
        ORD_POD(Replica Set
Order Pods)

        APIGW_POD1 -->|gRPC / HTTP L4| ORD_SVC
        APIGW_POD2 -->|gRPC / HTTP L4| ORD_SVC
        ORD_SVC -->|Least Connections| ORD_POD
        ORD_HPA -->|Escala preventiva| ORD_POD
    end

    subgraph "Inventory Namespace"
        INV_SVC[inventory-service
ClusterIP L4]
        INV_HPA(HPA: Umbral 75%
2-6 Pods)
        INV_POD(Replica Set
Inventory Pods)

        ORD_POD -->|Eventos (Broker SQS)| INV_SVC
        INV_SVC -->|Least Connections| INV_POD
        INV_HPA -->|Escala horizontal| INV_POD
        INV_POD -->|Invalidación Activa| Redis
    end

end

%% Capa de Persistencia Relacional
subgraph "Data Layer (AWS RDS PostgreSQL Multi-AZ)"
    CAT_DB[(db_catalog
Master - Escritura)]
    CAT_REP[(db_catalog
Read Replica - Lectura)]
    ORD_DB[(db_orders
Master - Transaccional)]

```

```

        INV_DB[(db_inventory
Master - Transaccional)]

        CAT_POD -->|CQRS (Sólo Lectura)| CAT_REP
        ORD_POD -->|Transacciones ACID| ORD_DB
        INV_POD -->|Transacciones ACID| INV_DB
        CAT_DB -->|Replicación Asíncrona nativa| CAT_REP
    end

%% Estilos Mermaid
classDef alb fill:#ff9900,stroke:#333,stroke-width:2px;
classDef svc fill:#326ce5,stroke:#fff,stroke-width:2px,color:#fff;
classDef hpa fill:#8e44ad,stroke:#fff,stroke-width:2px,color:#fff;
classDef pod fill:#1abc9c,stroke:#fff,stroke-width:2px,color:#fff;
classDef db fill:#34495e,stroke:#fff,stroke-width:2px,color:#fff;
classDef cache fill:#c0392b,stroke:#fff,stroke-width:2px,color:#fff;

class ALB alb;
class APIGW_SVC,CAT_SVC,ORD_SVC,INV_SVC svc;
class APIGW_HPA,CAT_HPA,ORD_HPA,INV_HPA hpa;
class APIGW_POD1,APIGW_POD2,CAT_POD,ORD_POD,INV_POD pod;
class CAT_DB,CAT_REP,ORD_DB,INV_DB db;
class Redis cache;

```